FICHE · S11 3 DANS VOTRE PROJET

Semaine 11 · Exercice 3 — Dans votre projet

Module ICT 291 · Développer des interfaces utilisateurs (UI)

Durée : 45 minutes · seul · parcours essentiel et avancé

Où sont les fichiers : dans le pack, dossier `02-exercices/s11/3-dans-votre-projet/depart/`

Mandat

Votre `index.html` de la semaine 9 montre des cartes écrites à la main. Votre `data.json` de la semaine 10 contient au moins 30 fiches. Votre mission : remplacer les cartes écrites à la main par celles de `data.json`, chargées par `fetch`, avec les quatre états de l'exercice 2, puis les mettre en ligne.

Pourquoi

C'est une des trois conditions de recevabilité de la note 3 : sans cartes chargées par `fetch` depuis un `data.json` personnel, le projet ne dépasse pas 3,5. À partir d'ici, chaque fiche ajoutée à `data.json` devient une carte sans toucher au HTML. La semaine 12 cherchera dans ces fiches.

Ce que vous saurez faire

- Relever ce que montre une carte, et le relier à vos étiquettes.
- Demander à l'IA d'adapter `afficherLieux` à **vos** cartes, puis vérifier ce qu'elle rend.
- Tester votre page avec Live Server, aux deux largeurs, puis en ligne.

Les mots

- **Live Server**, **`fetch`**, **`.then`**, **`.catch`**, **code de statut**, **onglet Réseau** : voir les exercices 1 et 2.
- **Racine du dépôt** : le dossier `m291-prenom` lui-même, là où se trouvent `index.html` et `data.json`.
- **`createElement`** : l'instruction qui fabrique un élément HTML vide, par exemple un `article`.
- **`textContent`** : le texte d'un élément. Ce qu'on y met s'affiche tel quel, jamais comme du HTML.
- **`innerHTML`** : le contenu d'un élément lu comme du HTML. Vous ne l'utilisez pas ici : une fiche qui contient `<` casserait la carte.

Fichiers fournis

Dans le dossier `depart/` de cet exercice :

- `mes-cartes.md` : le tableau à remplir à l'étape 2.
- `controle-json.html` : le même outil de contrôle qu'en semaine 10.

Vous avez aussi besoin de votre `app.js` de l'exercice 2, dans le dossier `s11-phare`.

Étapes

1. Contrôler `data.json`

Dans le Finder ou l'Explorateur, copiez `controle-json.html` sur votre bureau, pas dans votre dépôt. Ouvrez-le par un double-clic et collez-y tout le contenu de votre `data.json`.

C'est réussi si : l'outil dit que le JSON est valide, et compte au moins 30 fiches. Sinon, corrigez `data.json` avant d'aller plus loin : la suite n'affichera rien.

2. Relever une carte

Dans le Finder ou l'Explorateur, copiez `mes-cartes.md` à la racine de votre dépôt, à côté de `etiquettes.md`. **Dans VS Code**, ouvrez votre dépôt `m291-prenom`, puis `index.html`. Choisissez **une** carte écrite à la main et remplissez `mes-cartes.md`, ligne par ligne.

C'est réussi si : chaque texte de la carte a sa ligne, chaque ligne a une étiquette qui existe dans `etiquettes.md`, et la partie « Le conteneur » est remplie.

3. Vider la place des cartes

Dans `index.html`, copiez une carte entière dans `mes-cartes.md`, tout en bas : elle servira à l'étape 5. Puis supprimez toutes les cartes écrites à la main, sans toucher à l'élément qui les contenait. Donnez-lui l'`id` noté dans `mes-cartes.md`, par exemple `id="liste-cartes"`.

Juste au-dessus de cet élément, ajoutez :

```
<p id="etat" role="status"></p>
```

Et juste avant `</body>` :

```
<script src="app.js"></script>
```

Enregistrez.

C'est réussi si : la page s'ouvre avec son titre et ses styles, mais sans carte.

4. Apporter `app.js`

Dans le Finder ou l'Explorateur, copiez `app.js` de votre dossier `s11-phare` à la racine de votre dépôt, à côté de `index.html`. **Dans VS Code**, ouvrez-le. Si votre conteneur n'a pas l'`id` `liste-cartes`, changez-le dans la ligne `document.getElementById("liste-cartes")`. Puis faites un clic droit sur `index.html`, *Open with Live Server*.

C'est réussi si : la page montre une carte par fiche de votre `data.json`, même si elles ressemblent encore à celles de Phare, avec des trous. L'onglet Réseau montre `data.json` avec le code `200`.

5. Adapter la carte avec l'IA

Ouvrez votre assistant habituel. Collez la demande suivante, puis, à la suite, la carte copiée à l'étape 3, le contenu de `mes-cartes.md` et tout `app.js` :

```
Rends la fonction afficherLieux de ce fichier app.js pour qu'elle fabrique,
pour chaque fiche, exactement la carte HTML ci-dessous : mêmes éléments,
mêmes classes, même ordre. Mets à la place de chaque texte écrit à la main
l'étiquette indiquée dans mon tableau.
Avec createElement et textContent, sans innerHTML.
Garde les noms afficherLieux, tousLesLieux, conteneur et zoneEtat.
Ne change rien d'autre dans le fichier : ni fetch, ni .then, ni .catch,
ni les messages. Sans bibliothèque. Rends le fichier app.js complet.
```

Relisez ce qu'elle rend **avant** de le coller dans `app.js` : chaque ligne de `mes-cartes.md` doit y apparaître. Puis remplacez tout `app.js` par sa réponse et enregistrez.

C'est réussi si : les cartes ressemblent à celles de votre maquette de la semaine 9, et Ctrl+F (Cmd+F) `innerHTML` dans `app.js` ne trouve rien.

6a. Vos messages

Dans `app.js`, remplacez les trois messages de Phare par des phrases de votre application : chargement, erreur, liste vide. Par exemple « Chargement des lieux… » devient « Chargement des disques… ». Enregistrez.

C'est réussi si : vous pouvez lire à votre voisin vos trois phrases, et chacune parle de votre application, pas de Phare.

6b. Tester l'erreur

Dans `app.js`, remplacez `fetch("data.json")` par `fetch("data-absent.json")`. Enregistrez et regardez la page.

C'est réussi si : la page montre votre phrase d'erreur, et la console donne le code `404`.

6c. Tester le vide

Dans le Finder ou l'Explorateur, copiez `vide.json` depuis le dossier de l'exercice 2 à côté de votre `index.html`. **Dans `app.js`**, mettez `fetch("vide.json")`. Enregistrez et regardez la page. Puis remettez `fetch("data.json")`, enregistrez, et supprimez `vide.json` de votre dépôt avant de valider.

C'est réussi si : la page montre votre phrase de liste vide, puis vos cartes reviennent avec `fetch("data.json")`, et `vide.json` n'est plus dans votre dépôt.

7. Les deux largeurs

Dans le navigateur, regardez la page à 1440 px puis à 390 px, comme en semaine 9 (outils du navigateur, bouton des appareils).

C'est réussi si : aux deux largeurs, les cartes sont lisibles, rien ne dépasse à droite, et la console ne montre aucune ligne rouge.

8. Mettre en ligne

Dans VS Code, panneau **Contrôle de code source** : vérifiez que `app.js`, `index.html` et `mes-cartes.md` sont dans la liste. Écrivez le message `Affiche les cartes depuis data.json`, cliquez sur **Valider**, puis sur **Synchroniser les modifications**.

C'est réussi si : sur GitHub, `app.js` apparaît dans votre dépôt, et le dernier commit porte votre message.

9. Vérifier en ligne

Attendez une à deux minutes. Ouvrez l'adresse GitHub Pages de votre projet dans une **fenêtre de navigation privée**.

C'est réussi si : les cartes s'affichent en ligne, et l'onglet Réseau montre `data.json` avec le code `200`.

Réussi si

- Les cartes de `index.html` viennent toutes de `data.json` : il n'en reste aucune écrite à la main.
- `app.js` est à la racine, sans `innerHTML`, avec les noms `afficherLieux` et `tousLesLieux`.
- Le chargement, l'erreur et le vide affichent chacun une phrase de votre application.
- Lisible à 1440 et à 390 px, sans ligne rouge dans la console.
- En ligne, en navigation privée, les cartes s'affichent.

Si ça bloque

Ce que vous voyez	Ce qui se passe	Ce que vous faites
Aucune carte, et `404` pour `data.json`	`data.json` n'est pas à la racine, ou son nom diffère.	Placez-le à côté de `index.html`, nommé `data.json`, en minuscules.
Aucune carte, et `404` pour `app.js`	`app.js` est dans un sous-dossier, ou la ligne `<script>` dit autre chose.	`app.js` à la racine, et `<script src="app.js"></script>`.
`Cannot read properties of null`	`getElementById` ne trouve pas le conteneur ou `#etat`.	Comparez l'`id` de `index.html` et celui de `app.js`, lettre par lettre.
Des cartes avec des trous, ou `undefined`	Une étiquette du code n'est pas écrite comme dans `data.json`.	Comparez avec `etiquettes.md` : majuscules et accents comptent.

L'IA a réécrit `fetch` ou ajouté `async`	Elle a changé plus que la fonction demandée.	Refusez : « Ne change que la fonction afficherLieux. Rends le fichier complet. »
L'IA a écrit `innerHTML`	Elle a pris le chemin le plus court.	« Remplace innerHTML par createElement et textContent. Ne change rien d'autre. »
Les cartes s'affichent en local, pas en ligne	Le commit n'est pas synchronisé, ou GitHub Pages n'a pas fini.	Vérifiez le dernier commit sur GitHub, attendez deux minutes, rechargez en navigation privée.

Avec l’IA

L'IA écrit la fonction de vos cartes ; vous lui donnez la carte, les étiquettes et les limites. Sans votre tableau, elle invente des noms d'étiquettes qui ne sont pas dans votre `data.json`, et la page montre des cartes vides sans erreur. Avec votre tableau, vous vérifiez sa réponse ligne par ligne. Notez ce prompt dans votre journal IA, avec ce que vous avez corrigé à la main.

Pour les rapides

Parcours avancé : mêmes étapes, puis un ou plusieurs de ces points. Ils ne sont pas notés en plus.

- **Le compte.** Quand les cartes sont affichées, faites écrire dans `#etat` le nombre de fiches, par exemple « 30 disques ». Vérifiez : une fiche ajoutée à `data.json` fait passer le nombre à 31.
- **La fiche modifiée.** Changez une fiche de `data.json`, validez et synchronisez, puis vérifiez en ligne que sa carte a changé. C'est ce que vous montrerez à l'entretien.
- **Le zoom à 200 %.** Zoomez la page à 200 % : les cartes restent-elles lisibles ? Notez ce qui déborde, sans le corriger aujourd'hui.